

Segurança e proteção de dados

Os dados de um banco de dados são armazenados fisicamente em dispositivos de armazenamento, como discos magnéticos (HDDs) (Elmasri e Navathe, 2018) ou unidades de estado sólido (SSDs). Este capítulo apresenta a organização física do banco de dados, descreve o que é um catálogo de banco de dados e inclui instruções para concessão de acesso para permitir que usuários executem algumas instruções e outras não. Além disso, define instruções para privilégios de acesso a tabelas e visões.

Compreenderemos a necessidade de um banco de dados preservar a consistência dos dados na presença de algum tipo de falha. Muitas aplicações de banco de dados, como operações bancárias, requerem que as operações envolvendo manipulação de dados sejam atômicas - ou seja, indivisíveis, mesmo que diversas operações concorrentes ocorram simultaneamente. Por fim, o texto ainda nos apresenta as características oferecidas pelo SQL para especificar transações e garantir que as operações sejam, de fato, atômicas.

1 Organização do SGBD (Sistema de Gerenciamento de Banco de Dados) padrão SQL

Em um banco de dados relacional, a estrutura lógica envolve tabelas, índices e outros objetos, enquanto a estrutura física trata de como esses objetos estão armazenados no disco e como são acessados, dependendo do tipo de banco de dados.

1.1 Estrutura lógica

A tabela é o principal objeto lógico de um banco de dados relacional. Cada tabela é composta por linhas (também chamadas de registros ou tuplas) e colunas (também chamadas de campos ou atributos). Uma linha contém atributos de uma entidade. Por exemplo, uma linha da tabela FUNCIONARIO representa uma entidade de funcionário, e o valor de cada campo especifica um atributo que descreve um dado específico desse funcionário (por exemplo, nome, CPF, endereço, supervisor) (Elmasri e Navathe, 2018).

Para auxiliar no desempenho da consulta, o banco de dados utiliza índices que são estruturas de dados adicionais (como árvores B ou tabelas hash) para facilitar o acesso rápido aos dados. Cada índice contém uma chave de pesquisa (geralmente uma coluna da tabela) e um apontador para a localização física dos dados correspondentes.

As visões e procedimentos armazenados são mecanismos lógicos do banco de dados que não armazenam dados fisicamente, mas fornecem maneiras de acessar e manipular dados de forma eficiente.

1.2 Estrutura física

A estrutura física de um banco de dados começa com o armazenamento em blocos de disco ou páginas. Um bloco é a unidade de transferência de dados entre disco e memória (Elmasri e Navathe, 2018). Cada bloco possui um tamanho fixo e pode armazenar várias linhas da tabela. Segundo Elmasri e Navathe (2018), há duas formas de organizar os blocos de registros:

- *Organização espalhada* aproveita o espaço em disco não permitindo a ocorrência de espaços vazios, mas permitindo a ocorrência do pedaço inicial de um registro num bloco, apontando para o pedaço final do registro em outro bloco.
- *Organização não espalhada*, onde os registros não são espalhados em mais de um bloco, ou seja, cada bloco só possui registros completos; assim, para ler um registro, basta ler apenas um bloco do disco.

Sobre os arquivos do banco de dados são feitas as operações de recuperação (busca) e de modificação. São operações de modificação a inserção e remoção de registros e a alteração de valores de campos de registros do arquivo.

Uma organização de arquivo refere-se à organização dos dados do arquivo em registros, blocos e estruturas de acesso, distribuição e interligação na mídia de armazenamento. Um método de acesso diz respeito à forma como o SGBD localiza, lê e escreve dados no meio de armazenamento físico. Eles influenciam diretamente no desempenho das consultas e operações. Os principais métodos de acesso utilizados são:

- **Acesso sequencial ou pesquisa linear (do inglês, full table scan):** varre todos os registros, passando bloco por bloco do arquivo, do início ao fim. Esse método é utilizado quando não há índice disponível. Operações como COUNT(*), agregações ou varreduras com-

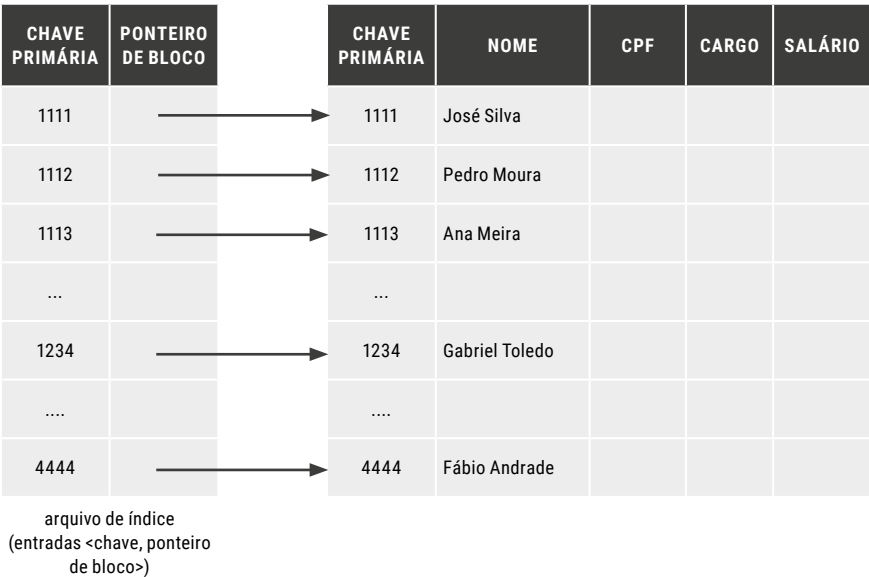
pletas utilizam o acesso sequencial. O exemplo 1 apresenta uma consulta cujo acesso é sequencial. Se a tabela FUNCIONARIO não tiver um índice em cidade, o banco de dados utilizará o método de acesso sequencial.

Exemplo 1 – Acesso sequencial

```
SELECT * FROM FUNCIONARIO WHERE cidade = 'São Paulo';
```

- **Acesso por índice (do inglês, Index Lookup):** usa o índice para localizar os dados em vez de varrer a tabela. Esse método é utilizado quando a consulta busca pela coluna indexada. É utilizado também para buscar por valores específicos, como chaves primárias ou colunas com índices. Este método faz a consulta por um índice da tabela (por uma estrutura do tipo árvore B). O apontador leva até o bloco de dados no disco. Apenas esse bloco é acessado. A figura 1 mostra uma organização de arquivos pelo método de acesso indexado pelo índice primário no campo chave primária.

Figura 1 – Acesso por índice





PARA SABER MAIS

Segundo Elmasri e Navathe (2018), existem diversos tipos de índices dos quais podem-se citar:

- Índice de agrupamento.
- Índice secundário.
- Índices multiníveis.
- Índices multiníveis dinâmicos usando B-trees e B⁺-trees;
- Índices em múltiplas chaves;
- Índices ordenado em múltiplos atributos;
- Índice de hash;
- Índice bitmap.

Para mais detalhes consulte o capítulo 8 do livro de Elmasri e Navathe (2018). Outra referência igualmente importante é o capítulo 11 do livro de Silberschatz, Korth e Sudarshan (2020).

2 Catálogo de banco de dados

O catálogo do banco de dados, também chamado de dicionário de dados (em inglês *data dictionary*) é um conjunto de tabelas que armazenam informações sobre o próprio banco de dados. Essa informação armazenada no catálogo são “dados sobre outros dados”, isso é chamado de metadados (Silberschatz, Korth e Sudarshan, 2020). O catálogo contém a descrição do esquema conceitual do banco de dados, do esquema interno, de todos os esquemas externos e dos mapeamentos entre os esquemas. Além disso, nele também são armazenados os módulos de otimização de consultas, de segurança e de autorização.

No caso dos bancos de dados relacionais, as informações devem ser armazenadas no catálogo no formato relacional, isto é, em forma de

tabelas de catálogo, que são geradas no momento da criação do banco de dados.

Entre as informações armazenadas no catálogo, incluem-se:

- Nomes das relações.
- Nomes dos atributos de cada relação.
- Domínio e tamanho do atributo.
- Relacionamentos, chaves estrangeiras e referências entre as tabelas.
- Estatísticas para otimização, tais como número de registro e tamanho das tabelas.

Descrições de visões e estruturas de armazenamento de índices também podem ser armazenadas no catálogo; procedimentos armazenados com informações sobre o nome do procedimento, parâmetros e corpo do procedimento; informações de autorização e segurança também; tudo na forma de relações, permitindo acesso generalizado ao catálogo via linguagem de consulta.

Muitos bancos de dados relacionais armazenam seus arquivos de catálogo como relações do SGBD. Mas, como há muito acesso ao catálogo, ele precisa ser implementado de forma eficiente.

3 Segurança e proteção de dados

Segundo Ramakrishnan e Gerhke (2011), existem três objetivos principais a serem considerados ao projetar uma aplicação de banco de dados segura:

- 1. Integridade:** a integridade do banco de dados diz respeito à necessidade de proteger as informações contra alterações impróprias, ou seja, criar, excluir ou modificar valores. A perda da integridade

pode ser intencional ou acidental e pode gerar fraudes ou tomada de decisão incorreta.

2. **Disponibilidade:** usuários autorizados não devem ser impedidos de acessar o sistema. O funcionário que deseja alterar as suas informações pessoais, como por exemplo o número do celular que mudou, deve ter permissão para fazê-lo.
3. **Sigilo:** as informações não devem estar disponíveis a usuários não autorizados. Por exemplo, um funcionário não deve ter permissão para visualizar as informações pessoais de outro funcionário.

Para alcançar esses objetivos, é importante definir uma política de segurança clara e consistente para descrever medidas de segurança que precisam ser aplicadas (Ramakrishnan e Gehrke, 2011). No contexto do sistema computacional, várias funções de segurança devem ser implementadas nas camadas de hardware, de sistema operacional e de SGBD.

Num SGBD multiusuário são oferecidas técnicas para permitir acesso a partes do BD ou para inibir o acesso a outras; desta forma, podem ser administradas as informações confidenciais. Para isso, o SGBD é dotado de um subsistema de autorização do BD contra acesso não autorizado, que consiste em conceder ou revogar privilégios. Contudo, além de se garantir a questão da confidencialidade deseja-se garantir a segurança e a integridade da informação, garantindo prevenção contra o acesso de pessoas não autorizadas na recuperação de informações e modificação do conteúdo do banco de dados (Elmasri e Navathe, 2018).

Uma solução bastante comum para o controle de acesso é o uso de contas e senhas. Outra técnica que se pode aplicar é a criptografia dos dados, comum quando há transmissão de dados por rede e quando se quer proteger porções sensíveis do banco de dados (Elmasri e Navathe, 2018).

O administrador do banco de dados (DBA) é a autoridade central no gerenciamento do banco de dados. Tem conta privilegiada: a conta de

sistema (Elmasri e Navathe, 2018). Quanto à segurança do banco de dados são atribuições do administrador a criação de contas e grupos de usuários com o fornecimento das respectivas senhas, a concessão e revogação de privilégios às contas criadas por meio de ações discricionárias (Ramakrishnan e Gehrke, 2011).

Uma pessoa ou grupo de pessoas é representada por uma conta cujo acesso é liberado por meio de um login e de uma senha, com a criação de uma tabela de login e senha e onde são definidas que operações se pode usar numa sessão de login. A sequência de operações executada é registrada no log do sistema, registrando também o terminal a partir do qual se conectou. A existência do log disponibiliza uma trilha de auditoria em que se pode fazer exame de acessos e operações executadas e, inclusive, verificar algum comportamento suspeito e possíveis violações de segurança.

No contexto de um SGBD relacional, a administração de privilégios é feita com o uso de instruções SQL, criadas para este propósito. Aqui, usuários e contas são tratados como sinônimos. Existem dois níveis de atribuição de privilégios, o nível de conta e o nível de relação. No nível de conta são definidos os privilégios para criar, remover, alterar (modificar) ou selecionar objetos como as tabelas ou as visões. Cada relação (real ou virtual) tem um proprietário. No nível de relação são oferecidos os privilégios selecionar e modificar.

Para realizar o controle de acesso discricionário o SQL fornece os comandos GRANT (para concessão de privilégios) e o comando REVOKE (para revogação de privilégios concedidos). Uma das primeiras concessões é o direito de criação de tabelas do banco de dados. Normalmente, este poder é concedido ao proprietário do banco de dados, que pode ser definido no comando de criação do esquema do banco de dados. Como exemplo, suponha o banco de dados Empresa, para dar permissão criar tabela utilize o comando GRANT, como mostra o exemplo 2.

Exemplo 2 – O comando GRANT para permitir ao usuário JSilva criar tabelas

```
CREATE SCHEMA Empresa AUTHORIZATION JSilva;  
ou  
GRANT CREATETAB TO JSilva;
```

Assim, o usuário JSilva pode criar tabelas, é proprietário do banco de dados e tem autoridade de concessão de privilégios a outros usuários. A partir daí, JSilva poderia instruir a permissão de inserir ou remover linhas na tabela FUNCIONARIO para o usuário CLopes, como mostra o exemplo 3.

Exemplo 3 – Permissão de inserir e remover linhas da tabela FUNCIONARIO a CLopes

```
GRANT INSERT, DELETE ON FUNCIONARIO TO CLopes;
```

O usuário CLopes adquiriu direitos de inserção e remoção na tabela FUNCIONARIO, bem como todos os direitos abaixo deste. Outro comando de concessão de privilégios que poderia ser executado por JSilva é o de propagar certo privilégio por meio de outro usuário, como demonstrado no exemplo 4.

Exemplo 4 – Permissão de consultar linhas da tabela FUNCIONARIO a FAlmeida com propagação

```
GRANT SELECT ON FUNCIONARIO TO FAlmeida WITH GRANT OPTION;
```

Agora, o usuário FAlmeida tem direito de fazer consultas na tabela FUNCIONARIO (mas não pode fazer atualização na tabela) e, ainda, adquiriu o privilégio de propagação de privilégios de consulta sobre esta tabela a outros usuários. Pode ser que, mais tarde, JSilva decida revogar de FAlmeida o privilégio de SELECT sobre FUNCIONARIO, como mostra o exemplo 5.

Exemplo 5 – Revogação de consulta sobre a tabela FUNCIONARIO a FAlmeida

```
REVOKE SELECT ON FUNCIONARIO TO FAlmeida;
```

Assim, esta revogação propaga-se automaticamente a todos os usuários que têm o privilégio de consulta sobre a tabela FUNCIONARIO que foi concedido por FAlmeida.

Agora, suponha que JSilva deseja devolver a FAlmeida uma capacidade limitada sobre a tabela FUNCIONARIO, por exemplo, FAlmeida somente pode consultar o nome, data de nascimento e endereço e somente tuplas do departamento número 5. No exemplo 6 podemos ver como fazer isso.

Exemplo 6 – Privilégio de consulta a uma visão

```
CREATE VIEW DADOS_FUNCIONARIO  
SELECT nome, datanasc, endereco  
FROM FUNCIONARIO  
WHERE ndepto = 5  
GRANT SELECT ON DADOS_FUNCIONARIO TO FAlmeida WITH GRANT  
OPTION;
```

Finalmente, JSilva deseja permitir que BSouza atualize somente o atributo salário de FUNCIONARIO, como mostra o exemplo 7.

Exemplo 7 – Privilégio de atualização sobre um atributo de FUNCIONARIO

```
GRANT UPDATE ON FUNCIONARIO (salario) TO BSouza;
```

4 Serializações, atomicidades e transações

Esta seção trata das transações, que são unidades atômicas de trabalho. Muitas aplicações de banco de dados requerem que as operações envolvendo os dados sejam atômicas, ou seja, realizadas de forma indivisível, mesmo que muitas operações concorrentes possam ocorrer simultaneamente. SQL oferece características e mecanismos que permitem especificar transações e garantir que as operações sejam transações.

Considere uma aplicação de transferência bancária entre contas.

- Operação 1: Um valor é retirado da conta origem.
- Operação 2: O mesmo valor é depositado na conta destino.

Para o usuário, ambas as operações devem ocorrer com sucesso. Se uma delas falhar, a transação toda deve ser desfeita, caso contrário, os dados a tornarão inconsistente.

Para o SGBD, uma transação é um programa em execução que forma a unidade lógica de processamento de banco de dados (Elmasri e Navathe, 2018). Ela inclui operações de inserção, remoção, alteração ou consulta.

Em uma transação todas as operações devem ser concluídas com sucesso, ou nenhuma operação deve ter efeito no banco, caso ocorra alguma falha. Esse comportamento exige módulos específicos para garantir a execução de forma completa e que seja possível executar várias transações de maneira simultânea, sem gerar inconsistência dos dados.

As transações possuem várias propriedades, normalmente chamadas de propriedades ACID (Elmasri e Navathe, 2018), são elas:

- **Atomicidade:** uma transação é atômica quando deve ser realizada completamente ou não ser realizada de maneira alguma.
- **Consistência:** uma transação deve preservar a consistência do banco de dados.
- **Isolamento:** garante que as transações sejam executadas sem a interferência de outras.
- **Durabilidade ou permanência:** garante que alterações ocorridas nas transações confirmadas serão mantidas no banco de dados.

A serialização em banco de dados refere-se ao processo de garantir que as transações sejam executadas de maneira segura e consistente, evitando conflitos e garantindo a integridade dos dados. A ideia é que, em um sistema com múltiplos usuários executando transações simultaneamente, as transações sejam executadas de modo que o estado final do banco de dados seja equivalente a uma execução sequencial dessas transações. Ou seja, a serialização garante que, ao final de todas as transações, o banco de dados permaneça em um estado consistente, como se cada transação fosse executada uma após a outra, em vez de simultaneamente.

Uma transação em SQL é uma coleção de uma ou mais operações envolvendo banco de dados e deve ser executada atomicamente, isto é, todas as operações são completamente executadas ou nenhuma delas é executada. Além disso, as transações devem ser executadas como se sequencialmente, ou seja, elas são serializáveis.

Segundo Elmasri e Navathe (2018), as transações iniciam quando qualquer comando SQL que consulta ou manipula o banco de dados se inicia. Não é necessário especificar qualquer comando inicial em SQL de início de transação. Entretanto, deve-se finalizar uma transação explicitamente de uma das duas formas:

- **COMMIT:** indica que todas as operações foram bem-sucedidas e podem ser gravadas no banco de dados.
- **ROLLBACK:** indica que ocorreu uma falha e que todas as alterações feitas até então devem ser revertidas.

Como exemplo, suponha a tabela CONTA que é criada no exemplo 8.

Exemplo 8 – tabela CONTA

```
CREATE TABLE CONTA(  
  id INT PRIMARY KEY,  
  nome VARCHAR(100),  
  saldo DECIMAL  
);  
  
INSERT INTO CONTA (1, 'José da Silva',  
1000.00);  
INSERT INTO CONTA (2, 'Maria Souza',  
1500.00);
```

E agora a transferência de R\$ 200,00 da conta 1 para a conta 2, ilustrada pelo exemplo 9.

Exemplo 9 – Transferência da conta 1 para a conta 2

```
UPDATE CONTA SET saldo = saldo - 200.0 WHERE id = 1;  
UPDATE CONTA SET saldo = saldo + 200.0 WHERE id = 2;  
COMMIT;
```

Mas se algo falhar no meio do caminho (por exemplo, a conta 2 não existir), é possível executar o comando ROLLBACK, o que desfaz as duas instruções de UPDATE.

Além dos comandos COMMIT e ROLLBACK, SQL possui um outro comando, denominado SET TRANSACTION, que determina o comportamento e a interação da transação com outras transações. Este comando descreve as ações que os comandos de uma transação podem executar, como uma transação deveria reagir a um conflito de travamento de registro ou tabela e descreve a visão do banco de dados dada a uma transação quando ela se relaciona com as ações por outras transações simultâneas.

As ações que os comandos de uma transação podem executar são definidas pelo modo de acesso da transação do banco de dados. O acesso pode ser apenas para leitura (READ ONLY), usado quando as ações são apenas de consulta às tabelas, ou para leitura e escrita (READ WRITE),

usado se o usuário deseja atualizar alguma tabela. Ambos os modos podem ser iniciados com o comando SET TRANSACTION, como mostra o exemplo 10.

Exemplo 10 – Comando SET TRANSACTION

```
SET TRANSACTION READ ONLY ;  
ou  
SET TRANSACTION READ WRITE ;
```

Um conflito de travamento pode ocorrer quando um registro ou uma tabela requisitada por uma transação estiver sendo usado por uma outra transação. SQL permite que o usuário decida se a transação que requisitou o recurso deve esperar a sua liberação ou utilizar o recurso com o estado antes da transação que o detém iniciar. O estado de espera é o comportamento padrão, pode ser explicitamente modificado com a opção NO WAIT present no comando SET TRANSACTION como mostra o exemplo 11.

Exemplo 11 – Comando SET TRANSACTION sem travamento

```
SET TRANSACTION [READ ONLY|READ WRITE] NO WAIT ;
```

Considerações finais

Este capítulo apresentou aspectos lógicos e físicos da organização de um banco de dados. Sobre os aspectos físicos, vimos como os arquivos do banco de dados são organizados e as maneira como são acessados, por meio de uma varredura sequencial ou acesso por índices. Após isso, vimos o catálogo do banco de dados, também conhecido por dicionário de dados, que contém um conjunto de tabelas com o registro de todos os objetos do banco de dados, como tabelas, visões, procedimentos armazenados, entre outros.

Na sequência, o texto apresentou aspectos de segurança e comandos SQL para conceder ou revogar privilégios de consultas, inserções, remoções ou alterações sobre as tabelas ou visões de banco de dados.

Compreendemos as transações e suas propriedades, que são propriedades ACID, acrônimo para atomicidade, consistência, isolamento e durabilidade. E finalizamos com o conceito de transações serializáveis, e como o SGBD trata as transações com os comandos SQL.

Referências

ELMASRI, Ramez; NAVATHE, Shamkant B. **Sistemas de banco de dados**. 7. ed. São Paulo: Pearson, 2018.

RAMAKRISHNAN, Raghu; GEHRKE, Johannes. **Sistema de gerenciamento de banco de dados**. 3. ed. Porto Alegre: AMGH, 2011.

SILBERSCHATZ, Abraham; KORTH, Henry F; SUDARSHAN, S. **Sistema de Banco de Dados**. 7. ed. Rio de Janeiro: GEN LTC, 2020.

Sobre o autor

Leonardo Takuno possui graduação em ciência da computação pela Universidade Federal de Mato Grosso do Sul – UFMS (1997), especialização em análise e desenvolvimento de sistemas pela UFMS (2000) e mestrado em engenharia da computação pela Escola Politécnica da Universidade de São Paulo – USP (2004). Atualmente, é professor na Faculdade Impacta de Tecnologia (FIT), no Centro Universitário Senac – SP e na Universidade Presbiteriana Mackenzie. O autor possui 20 anos de experiência em desenvolvimento de software, atuou com desenvolvimento de sistemas para comércio, área médica e veterinária, área jurídica, rastreamento de veículos e desenvolvimento de aplicativos mobile da plataforma Apple. Na docência, ministra disciplinas relacionadas a algoritmos e programação, banco de dados, análise de sistemas, arquitetura de software, entre outras.

Link para o Currículo Lattes: <http://lattes.cnpq.br/4246633386839100>

